TYPOSTÉROÏDES

Séance de 2 h

TD1 – La boucle de jeu et les mots-astéroïdes

Un jeu de frappe en Python avec pygame

Version corrigée

Objectifs de la séance

À la fin de cette séance, vous aurez codé la première version de *Typostéroïdes* : un vaisseau au centre de l'écran, des mots qui naissent sur les bords et foncent vers lui, et une partie perdue quand le bouclier a été touché trois fois.

- comprendre **comment fonctionne pygame** ;
- écrire une **boucle de jeu** (événements, mise à jour, dessin) ;
- rendre un mouvement indépendant des FPS avec `dt` ;
- utiliser des **classes** : attributs, méthodes, `self` ;
- manipuler des **vecteurs** (`pygame.Vector2`) ;
- gérer une **liste d'objets** qui naissent et disparaissent ;
- détecter une **collision** avec une distance ;
- travailler **par petites étapes testées**, comme un développeur de jeu.

Table des matières

1 Le projet « Typostéroïdes » 1

2 Comment fonctionne pygame 2

2.1 Qu'est-ce que pygame ? . . . 2
2.2 Le squelette de *tout* programme pygame . . . 2
2.3 La boucle de jeu . . . 3
2.4 Le temps : `Clock` et `dt` . . . 3
2.5 Surfaces, repère et rectangles . . . 3
2.6 Dessiner et écrire . . . 4
2.7 Les événements . . . 4
2.8 Les vecteurs `pygame.Vector2` . . . 4

3 Construire le jeu, étape par étape 5

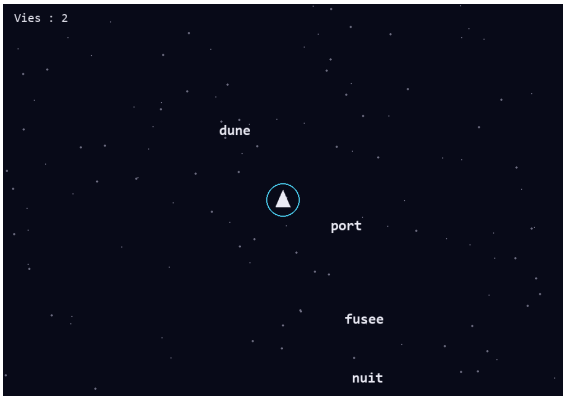
4 Pour aller plus loin (si vous avez fini) 11

5 Backlog : tout ce qu'on peut ajouter au jeu 12

1 Le projet « Typostéroïdes »

Le jeu s'inspire de *Glyphica : Typing Survival* et de *ZType* : le joueur pilote un vaisseau **immobile** au centre de l'écran. Des mots arrivent de tous les côtés, de plus en plus vite. Pour détruire un mot, il faut le **taper au clavier** le plus vite possible. Si un mot atteint le vaisseau, le joueur perd une vie.

Une contrainte : tout le jeu tient dans **un seul fichier `.py`**. On l'organise donc avec soin : constantes en haut, une classe par type d'objet, puis la classe `Jeu` qui orchestre le tout.



Le résultat attendu à la fin du TD1.

Organisation du module (14 h)

Séance	Durée	Contenu
TD1	2 h	Fonctionnement de pygame, boucle de jeu, joueur, mots en mouvement, collisions, vies *(cette fiche)*
TD2	2 h	Menu et game over, frappe et ciblage, lasers, score, difficulté progressive
TP1	4 h	Effets visuels et sonores, dictionnaire externe, accents
TP2	3 h	Ennemis spéciaux, bonus, boss, vagues et améliorations
TP3	3 h	Sauvegarde des scores, pause, options, finition et rendu

Le fichier `td2_correction.py` distribué à la fin du TD2 servira de base commune à tous les TP.

Fichiers fournis

— `td1_eleve.py` : le squelette à compléter. Il se lance déjà, mais il manque des morceaux, signalés par des commentaires `TODO n`. **Le numéro du TODO est le numéro de l'étape** de cette fiche où on le complète.

— Cette fiche : la partie 2 explique pygame, la partie 3 vous guide étape par étape.

2 Comment fonctionne pygame

2.1 Qu'est-ce que pygame ?

`pygame` est une bibliothèque Python pour écrire des jeux 2D. Elle s'appuie sur *SDL*, une bibliothèque C utilisée par de nombreux jeux professionnels, qui parle à la carte graphique, au clavier, à la souris et à la carte son. pygame est découpée en **modules** ; nous en utiliserons quelques-uns :

Module	Rôle
`pygame.display`	la fenêtre et l'affichage à l'écran
`pygame.event`	les événements : clavier, souris, fermeture de la fenêtre
`pygame.draw`	dessiner des formes : cercles, lignes, rectangles, polygones
`pygame.font`	écrire du texte
`pygame.time`	le temps : l'horloge `Clock` et le cadencement
`pygame.math`	les vecteurs `Vector2` (aussi accessibles par `pygame.Vector2`)
`pygame.mixer`	le son (au TP1)
`pygame.image`	charger des images `.png`, `.jpg` (au TP1)

2.2 Le squelette de *tout* programme pygame

Tous les jeux pygame, du plus simple au plus complexe, ont cette forme. Lisez-le ligne par ligne : tout le TD1 en est un développement.

```
import pygame

pygame.init()                                       # 1. démarrer tous les modules
ecran = pygame.display.set_mode((800, 600))         # 2. ouvrir la fenêtre -> une Surface
pygame.display.set_caption("Mon jeu")
horloge = pygame.time.Clock()

x = 0.0                                             # l'"état" du jeu : ici une position
en_cours = True
while en_cours:                                     # 3. LA BOUCLE DE JEU
    dt = horloge.tick(60) / 1000                    #    a) cadencer à 60 images/s

    for evenement in pygame.event.get():            #    b) ÉVÉNEMENTS
        if evenement.type == pygame.QUIT:           #       clic sur la croix
            en_cours = False

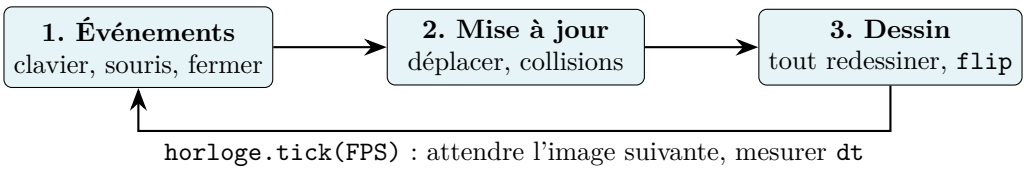
    x = x + 100 * dt                                #    c) MISE À JOUR (100 px/s)

    ecran.fill((0, 0, 0))                           #    d) DESSIN : effacer...
    pygame.draw.circle(ecran, (255, 0, 0), (x, 300), 20)   # ...dessiner...
    pygame.display.flip()                           #    ...et afficher

pygame.quit()                                       # 4. fermer proprement
```

2.3 La boucle de jeu

Un jeu vidéo est un programme qui **ne s'arrête jamais de tourner**. Environ 60 fois par seconde, il répète trois étapes :



— **Événements** : pygame range dans une *file d'attente* tout ce qui s'est passé depuis la dernière image. `pygame.event.get()` vide cette file et renvoie la liste des événements.

— **Mise à jour** : on fait évoluer l'état du jeu (positions, vies, score…). *Rien n'est affiché à cette étape.*

— **Dessin** : on efface tout (`fill`), on redessine *tout* dans l'ordre (du fond vers le premier plan), puis `pygame.display.flip()` affiche l'image d'un coup.

Pourquoi tout redessiner à chaque image ?

L'écran n'est pas une feuille où les objets « vivent » : c'est une **image** (une grille de pixels). Un cercle dessiné n'est plus un objet, ce sont des pixels colorés. Pour qu'il « bouge », on efface tout et on le redessine un peu plus loin, 60 fois par seconde : c'est le principe du dessin animé. Pygame dessine dans une image cachée (*back buffer*) et `flip()` l'échange avec l'image visible, ce qui évite de voir l'image se construire (*double buffering*).

2.4 Le temps : `Clock` et `dt`

`horloge.tick(FPS)` fait deux choses : il **attend** ce qu'il faut pour ne pas dépasser `FPS` images par seconde, et il **renvoie le temps écoulé** depuis l'appel précédent, en *millisecondes*. Divisé par 1000, c'est le *delta time* `dt`, en secondes.

Si l'on déplaçait un mot de 1 pixel par image, il irait deux fois plus vite sur un ordinateur à 120 FPS que sur un ordinateur à 60 FPS. En exprimant les vitesses en **pixels par seconde** et en multipliant par `dt`, le mouvement est le même partout :

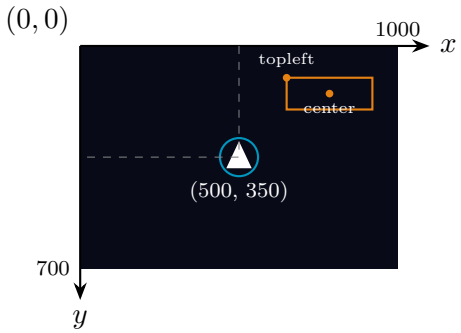
$$\text{déplacement pendant une image} = v \times dt = 55 \text{ px/s} \times \tfrac{1}{60} \text{ s} \approx 0{,}92 \text{ px.}$$

2.5 Surfaces, repère et rectangles

Dans pygame, toute image est une **`Surface`** : la fenêtre (`ecran`), une image chargée, un texte rendu. **`blit`** copie une Surface sur une autre : `ecran.blit(image, (x, y))`.

Attention au repère : l'origine $(0, 0)$ est **en haut à gauche** et **l'axe y est orienté vers le bas**. Le centre d'une fenêtre 1000×700 est $(500, 350)$. Un point « plus haut » a un y **plus petit**.

Un **`Rect`** est un rectangle $(x, y, \text{largeur}, \text{hauteur})$ qui sait se placer par ses points remarquables : `topleft`, `center`, `midleft`, `bottomright`… Très pratique pour centrer un texte : `rect = image.get_rect(center=(500, 350))`.



2.6 Dessiner et écrire

Aide-mémoire : dessin et texte

```
ecran.fill(couleur)                                          # tout remplir
pygame.draw.circle(ecran, couleur, centre, rayon, epaisseur) # 0 = plein
pygame.draw.line(ecran, couleur, debut, fin, epaisseur)
pygame.draw.rect(ecran, couleur, rect, epaisseur)
pygame.draw.polygon(ecran, couleur, [p1, p2, p3])            # triangle plein

police = pygame.font.SysFont("consolas", 26, bold=True)      # à créer UNE fois
image = police.render("bonjour", True, (255, 255, 255))      # texte -> Surface
ecran.blit(image, image.get_rect(center=(500, 350)))         # coller, centré
```

Une couleur est un tuple `(rouge, vert, bleu)`, chaque valeur entre 0 et 255.

2.7 Les événements

Chaque événement a un `type`. Les plus utiles :

Type	Quand ?	Attributs utiles
`pygame.QUIT`	clic sur la croix de la fenêtre	—
`pygame.KEYDOWN`	une touche est enfoncée	`key` (`pygame.K_ESCAPE...`), `unicode` (`"a"`)
`pygame.KEYUP`	une touche est relâchée	`key`
`pygame.MOUSEBUTTONDOWN`	clic de souris	`pos`, `button`

`evenement.key` identifie la *touche physique* ; `evenement.unicode` donne le *caractère* produit en tenant compte du clavier (AZERTY ou QWERTY) et de Maj. Pour un jeu de frappe, c'est `unicode` qui nous intéressera au TD2.

2.8 Les vecteurs `pygame.Vector2`

Aide-mémoire : vecteurs

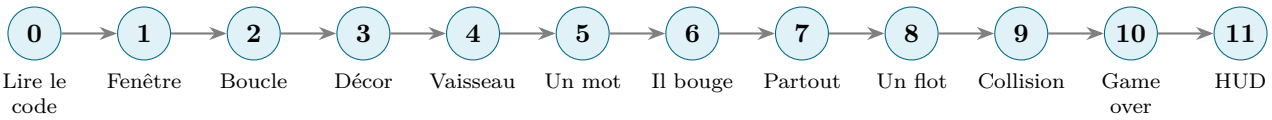
```
p = pygame.Vector2(500, 350)        # p.x = 500.0, p.y = 350.0
q = pygame.Vector2(800, 50)
q - p                  # vecteur qui va de p vers q : Vector2(300, -300)
(q - p).length()       # sa longueur (norme) : 424.26...
(q - p).normalize()    # même direction, longueur 1 : Vector2(0.707, -0.707)
p.distance_to(q)       # distance entre deux points : 424.26...
v * 3, v + w           # multiplication par un nombre, addition
v.rotate(90)           # rotation de 90 degrés (utile au TD2)
x, y = p               # "déballage" en deux nombres
```

Les erreurs pygame les plus fréquentes

- **La fenêtre « ne répond pas »** : la file d'événements n'est pas vidée à chaque tour (`pygame.event.get()` manquant).
- **Rien ne s'affiche** : `pygame.display.flip()` oublié.
- **Les objets laissent des traînées** : `ecran.fill(...)` oublié au début du dessin.
- **Le jeu rame** : on crée une police (`SysFont`) ou on charge une image *dans* la boucle. On les crée une seule fois, dans `__init__`.
- **Un objet est dessiné mais caché** : l'ordre de dessin compte, le dernier dessiné est au-dessus.

3 Construire le jeu, étape par étape

Un développeur de jeu n'écrit jamais 300 lignes d'un coup pour les tester à la fin. Il avance par **toutes petites étapes** : *ajouter une chose → lancer le jeu → vérifier que ça marche → passer à la suite*. Si quelque chose casse, on sait que c'est dans les dernières lignes écrites. Respectez cet ordre et **lancez le jeu à chaque encadré vert « Testez »**.



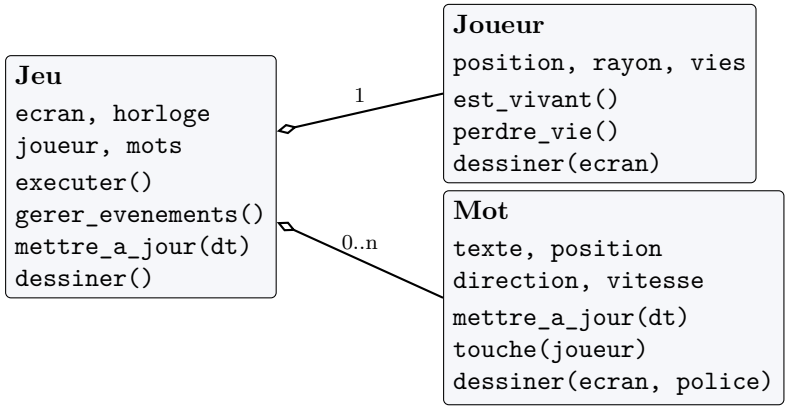
Étape 0 – Lire le squelette 10 min

Installez `pygame` puis vérifiez qu'il fonctionne :

```
pip install pygame
python -m pygame.examples.aliens      # un petit jeu de démonstration doit s'ouvrir
```

Ouvrez `td1_eleve.py`. Le fichier est découpé en cinq parties :

1. Constantes
2. `Joueur`
3. `Mot`
4. `Jeu`
5. Point d'entrée



Questions :

1. Où sont définies la taille de la fenêtre et la vitesse des mots ? Pourquoi les écrit-on en MAJUSCULES ?
2. Combien d'objets `Joueur` et combien d'objets `Mot` existeront pendant une partie ?
3. Quelle méthode remet le jeu à zéro ? Pourquoi est-elle séparée de `__init__` ?
4. Que contient la ligne `if __name__ == "__main__":` et à quoi sert-elle ?

Correction

1. En haut du fichier (`LARGEUR`, `HAUTEUR`, `VITESSE_MOT`). Les majuscules sont une *convention* Python pour les constantes : des réglages qu'on ne modifie pas pendant l'exécution, regroupés à un seul endroit pour équilibrer le jeu.
2. Un seul `Joueur` ; un nombre variable de `Mot`, stockés dans la liste `self.mots`.
3. `nouvelle_partie()`. `__init__` fait le travail « une fois pour toutes » (fenêtre, polices, étoiles) ; `nouvelle_partie` réinitialise seulement ce qui change d'une partie à l'autre, ce qui permet de rejouer sans recréer la fenêtre.
4. `Jeu().executer()` : on crée le jeu et on lance la boucle. Le test n'est vrai que si le fichier est *exécuté* (et pas importé par un autre fichier).

Étape 1 – Ouvrir une fenêtre 5 min

Ce code est **déjà écrit** dans `Jeu.__init__`. Retrouvez chaque ligne dans le squelette de la partie 2 :

```
pygame.init()
self.ecran = pygame.display.set_mode((LARGEUR, HAUTEUR))   # la Surface-fenêtre
pygame.display.set_caption(TITRE)
self.horloge = pygame.time.Clock()
self.police_mot = pygame.font.SysFont("consolas", 26, bold=True)  # créée UNE fois
```

✓ Testez : lancez le jeu, vous devez voir…

Lancez `td1_eleve.py` : la fenêtre s'ouvre… et se referme aussitôt. **C'est normal** : `executer` ne contient pas encore de boucle, le programme arrive directement à `pygame.quit()`.

Étape 2 – La boucle de jeu TODO 2 — 15 min

Complétez `executer` : tant que `self.en_cours` est vrai, calculez `dt` **en secondes** puis appelez dans l'ordre `gerer_evenements()`, `mettre_a_jour(dt)` et `dessiner()`.
Lisez ensuite `gerer_evenements` (déjà écrite) : quels événements mettent `self.en_cours` à `False` ?

✓ Testez : lancez le jeu, vous devez voir…

La fenêtre reste ouverte et affiche un ciel étoilé. `Échap` ou la croix ferment la fenêtre.

Correction

```
def executer(self):
    while self.en_cours:
        dt = self.horloge.tick(FPS) / 1000      # ms -> secondes
        self.gerer_evenements()
        self.mettre_a_jour(dt)
        self.dessiner()
    pygame.quit()
```

`QUIT` (la croix) et `KEYDOWN` avec `K_ESCAPE` arrêtent la boucle.

Étape 3 – Le décor — 5 min

Le fond étoilé est déjà écrit. Dans `__init__`, les étoiles sont tirées au hasard **une seule fois** et stockées dans une liste de tuples (x, y, taille) :

```
self.etoiles = [(random.randint(0, LARGEUR), random.randint(0, HAUTEUR),
                 random.randint(1, 2)) for _ in range(NB_ETOILES)]
```

puis `dessiner_fond` les redessine à chaque image.

1. Passez `NB_ETOILES` à 500, puis remettez 120.
2. Que verrait-on si l'on tirait les positions au hasard *dans* `dessiner_fond` ?

Correction

Les étoiles changeraient de place 60 fois par seconde : un « grésillement » illisible. Tout ce qui est aléatoire mais *permanent* se tire une fois, puis on le mémorise.

Étape 4 – Le vaisseau TODO 4 — 10 min

Complétez `Joueur.dessiner` : le bouclier est un cercle `CYAN` d'épaisseur 2 et de rayon `self.rayon` ; le vaisseau est un triangle `BLANC` dont la pointe est 18 px **au-dessus** du centre et les deux autres sommets 12 px en dessous, à ± 13 px.

Rappel : « au-dessus » $\Rightarrow$ y plus petit !

✓ Testez : lancez le jeu, vous devez voir…

Le vaisseau, pointe vers le haut, entouré de son bouclier, au centre de l'écran.

Correction

```python
def dessiner(self, ecran):
    x, y = self.position
    pygame.draw.circle(ecran, CYAN, self.position, self.rayon, 2)
    pointe = (x, y - 18)          # y - 18 : AU-DESSUS (l'axe y descend)
    gauche = (x - 13, y + 12)
    droite = (x + 13, y + 12)
    pygame.draw.polygon(ecran, BLANC, [pointe, gauche, droite])
```

Étape 5 – Un premier mot, immobile — 5 min

Avant de gérer des dizaines de mots, on en affiche **un seul**. La méthode `Mot.dessiner` est fournie : elle *rend* le texte en image puis le colle centré sur `self.position`. Ajoutez **temporairement** à la fin de `nouvelle_partie` :

```python
self.mots.append(Mot("bonjour", self.joueur.position, VITESSE_MOT))    # PROVISOIRE
```

✓ Testez : lancez le jeu, vous devez voir...

Un morceau du mot « bonjour » dans le coin en haut à gauche. Pourquoi là ? Parce que `Mot.__init__` place pour l'instant tous les mots en $(0, 0)$ (le `TODO 7` n'est pas fait) et que le texte est *centré* sur ce point.

Étape 6 – Faire bouger le mot `TODO 6a` `TODO 6b` `TODO 6c` — 20 min

Un `Mot` a besoin d'une **direction** et d'une **vitesse**.

La direction : un vecteur unitaire. Le vecteur qui va de P (le mot) vers C (le centre) est $\overrightarrow{PC} = C - P$. Sa longueur est la distance PC, qui dépend de l'endroit où le mot est né. On le **normalise** pour ne garder que la *direction* :

$$\vec{u} = \frac{\overrightarrow{PC}}{\|\overrightarrow{PC}\|} \quad \Longrightarrow \quad \texttt{direction = (centre - self.position).normalize()}$$

Le vecteur vitesse est alors $\vec{v} = \vec{u} \times \text{vitesse}$, en pixels par seconde.

Le mouvement : intégration pas à pas. À chaque image, la position avance de $\vec{v} \times dt$ (c'est la *méthode d'Euler*, que vous reverrez en physique) :

$$\vec{p}(t + dt) = \vec{p}(t) + \vec{v}\,dt \quad \Longrightarrow \quad \texttt{self.position += self.vitesse * dt}$$

1. `TODO 6a` : dans `Mot.__init__`, calculez `self.direction` puis `self.vitesse`.
2. `TODO 6b` : complétez `Mot.mettre_a_jour(dt)`.
3. `TODO 6c` : dans `Jeu.mettre_a_jour`, mettez à jour chaque mot de `self.mots`.

✓ Testez : lancez le jeu, vous devez voir...

« bonjour » glisse en diagonale du coin haut-gauche vers le vaisseau, le traverse, puis continue sa route et sort de l'écran (on gèrera la collision à l'étape 9).

Correction

```python
# Mot.__init__
self.direction = (centre - self.position).normalize()
self.vitesse = self.direction * vitesse
```

```python
# Mot.mettre_a_jour
def mettre_a_jour(self, dt):
    self.position += self.vitesse * dt

# Jeu.mettre_a_jour
for mot in self.mots:
    mot.mettre_a_jour(dt)
```

Exercice 1 – Comprendre les vecteurs — 5 min

1. Que se passerait-il si l'on oubliait `.normalize()` ?
2. Pourquoi écrit-on `centre - self.position` et pas `self.position - centre` ?
3. Un mot naît à $P = (1040, 350)$, avec $C = (500, 350)$ et une vitesse de 55 px/s. Calculez $\vec{u}$, $\vec{v}$ et le temps qu'il met pour atteindre le centre.

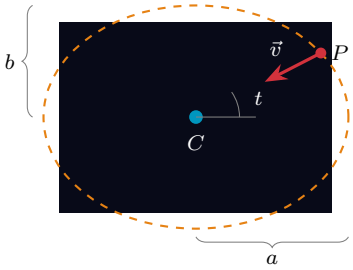
Correction

1. La vitesse serait proportionnelle à la distance de départ : avec $\vec{v} = 55\,\overrightarrow{PC}$, *tous* les mots atteindraient le centre en 1/55 s, c'est-à-dire presque instantanément.
2. $\overrightarrow{PC} = C - P$ va *de* P *vers* C ; l'inverse ferait fuir les mots.
3. $\overrightarrow{PC} = (-540, 0)$, $\vec{u} = (-1, 0)$, $\vec{v} = (-55, 0)$ px/s, temps $= 540/55 \approx 9{,}8$ s.

Étape 7 – Naître de partout `TODO 7` — 15 min

On veut que les mots arrivent de *tous les côtés*, juste hors de la zone visible. On choisit un angle t au hasard dans $[0, 2\pi[$ et on place le mot sur l'ellipse de centre C et de demi-axes a et b :

$$\begin{cases} x = c_x + a\cos t \\ y = c_y + b\sin t \end{cases} \qquad \begin{aligned} a &= \tfrac{\texttt{LARGEUR}}{2} + \texttt{MARGE_APPARITION} \\ b &= \tfrac{\texttt{HAUTEUR}}{2} + \texttt{MARGE_APPARITION} \end{aligned}$$



(sur l'écran, y descend : l'angle tourne donc dans le sens horaire)

Avec un *cercle*, les mots naîtraient bien plus loin à gauche et à droite qu'en haut et en bas, car la fenêtre n'est pas carrée : l'ellipse « épouse » le rectangle.

En Python : `random.uniform(0, 2 * math.pi)`, `math.cos`, `math.sin`.

`TODO 7` : dans `Mot.__init__`, remplacez `pygame.Vector2(0, 0)` par le point de l'ellipse. Puis remplacez votre ligne provisoire de l'étape 5 par une boucle qui ajoute 8 mots.

✓ Testez : lancez le jeu, vous devez voir…

Huit mots partent de huit endroits différents autour de l'écran et convergent vers le vaisseau. Relancez plusieurs fois : les positions changent. **Supprimez ensuite le code provisoire.**

Correction

```python
angle = random.uniform(0, 2 * math.pi)
a = LARGEUR / 2 + MARGE_APPARITION
b = HAUTEUR / 2 + MARGE_APPARITION
self.position = pygame.Vector2(centre.x + a * math.cos(angle),
                               centre.y + b * math.sin(angle))
```

Étape 8 – Un flot de mots `TODO 8a` `TODO 8b` 15 min

Pour créer un mot toutes les `DELAI_APPARITION` secondes, on utilise un **minuteur** : une variable à laquelle on ajoute `dt` à chaque image, et qu'on compare au délai.

```
self.minuteur_apparition += dt
if self.minuteur_apparition >= DELAI_APPARITION:
    self.minuteur_apparition -= DELAI_APPARITION    # et non "= 0" : voir l'astuce
    ...                                             # c'est l'heure : on agit
```

1. `TODO 8a` : `faire_apparaitre_mot` choisit un texte au hasard (`random.choice(MOTS)`), crée un `Mot` qui vise `self.joueur.position` et l'ajoute à `self.mots`.
2. `TODO 8b` : écrivez le minuteur dans `Jeu.mettre_a_jour`.
3. Ajoutez temporairement `print(len(self.mots))` dans `mettre_a_jour`. Que constatez-vous au bout d'une minute ? Pourquoi est-ce un problème ?

Pourquoi `-=` plutôt que `= 0` ?

Le minuteur dépasse presque toujours un peu le délai (par exemple 2,013 s). Avec `= 0` on *perd* ces 13 ms à chaque apparition et le rythme dérive. Avec `-=` on garde ce reste et le rythme moyen est exact. Remarquez aussi que `nouvelle_partie` démarre le minuteur « plein » pour qu'un premier mot apparaisse immédiatement.

✓ Testez : lancez le jeu, vous devez voir…

Un nouveau mot toutes les 2 secondes, venant d'un côté au hasard.

Correction

```
def faire_apparaitre_mot(self):
    texte = random.choice(MOTS)
    self.mots.append(Mot(texte, self.joueur.position, VITESSE_MOT))

# dans Jeu.mettre_a_jour(self, dt) :
self.minuteur_apparition += dt
if self.minuteur_apparition >= DELAI_APPARITION:
    self.minuteur_apparition -= DELAI_APPARITION
    self.faire_apparaitre_mot()
```

Question 3 : la liste grossit sans fin (30 mots par minute), car les mots traversent le vaisseau et continuent hors de l'écran sans jamais être supprimés. Le jeu consomme de plus en plus de mémoire et de calcul : c'est une *fuite*. L'étape suivante la corrige.

Étape 9 – Collision avec le bouclier `TODO 9a` `TODO 9b` `TODO 9c` 15 min

Le bouclier est un cercle de centre J (le joueur) et de rayon r. Un mot de position M le touche si la distance JM est inférieure à r :

$$JM = \sqrt{(x_M - x_J)^2 + (y_M - y_J)^2} < r \qquad \Longrightarrow \qquad \texttt{m.distance_to(j) < r}$$

1. `TODO 9a` : `Mot.touche(joueur)` renvoie `True` si le mot est dans le bouclier.
2. `TODO 9b` : `Joueur.perdre_vie()` retire une vie.
3. `TODO 9c` : dans `Jeu.mettre_a_jour`, construisez la liste `survivants` : un mot qui touche le joueur lui retire une vie et n'est pas gardé. Remplacez ensuite `self.mots` par `survivants`.

Piège classique : supprimer pendant qu'on parcourt

Le code ci-dessous semble correct mais **oublie des éléments** :

```
nombres = [1, 2, 2, 3]
for n in nombres:
    if n == 2:
        nombres.remove(n)
print(nombres)        # [1, 2, 3]  : un 2 a survécu !
```

Quand on retire un élément, les suivants se décalent d'une case et la boucle « saute » celui qui vient prendre sa place. Solution : **construire une nouvelle liste** avec les éléments à garder, puis remplacer l'ancienne.

✓ Testez : lancez le jeu, vous devez voir...

Les mots disparaissent en touchant le bouclier. Le `print(len(self.mots))` de l'étape 8 reste stable (vous pouvez maintenant le retirer). Le nombre de vies baisse (vérifiez avec un `print(self.joueur.vies)` provisoire) ; il devient négatif... c'est l'étape suivante.

Correction

```
def touche(self, joueur):                          # dans Mot
    return self.position.distance_to(joueur.position) < joueur.rayon

def perdre_vie(self):                              # dans Joueur
    self.vies -= 1

survivants = []                                    # dans Jeu.mettre_a_jour
for mot in self.mots:
    if mot.touche(self.joueur):
        self.joueur.perdre_vie()
    else:
        survivants.append(mot)
self.mots = survivants
```

Étape 10 – Game over et rejouer TODO 10a TODO 10b 10 min

1. TODO 10a : `Joueur.est_vivant()` renvoie `True` s'il reste au moins une vie.
2. TODO 10b : à la fin de `Jeu.mettre_a_jour`, si le joueur n'est plus vivant, passez `self.game_over` à `True`.
3. Lisez le début de `mettre_a_jour` et `gerer_evenements` : qu'est-ce qui « fige » le jeu après un game over ? Qu'est-ce qui permet de rejouer ?

✓ Testez : lancez le jeu, vous devez voir...

Au troisième mot reçu, « GAME OVER » s'affiche et les mots se figent. `R` relance une partie avec 3 vies et un écran vide.

Correction

```
def est_vivant(self):
    return self.vies > 0          # une comparaison vaut déjà True ou False

if not self.joueur.est_vivant():  # fin de Jeu.mettre_a_jour
    self.game_over = True
```

Le `if self.game_over: return` en tête de `mettre_a_jour` empêche toute évolution ; le dessin, lui, continue (on voit la scène figée). La touche `R` appelle `nouvelle_partie()` qui recrée un joueur et vide la liste.

Étape 11 – Le HUD : afficher les vies `TODO 11` 5 min

Le HUD (*Head-Up Display*) regroupe les informations dessinées par-dessus le jeu. Dans `dessiner_hud`, affichez `Vies : 3` en haut à gauche (position $(20, 15)$), avec une f-string : `f"Vies : {self.joueur.vies}"`.

Question : dans `dessiner`, pourquoi le HUD est-il dessiné *après* les mots ?

✓ Testez : lancez le jeu, vous devez voir...

« Vies : 3 » en haut à gauche, qui diminue à chaque choc.

Correction

```python
def dessiner_hud(self):
    texte = f"Vies : {self.joueur.vies}"
    image = self.police_hud.render(texte, True, BLANC)
    self.ecran.blit(image, (20, 15))
```

Le dernier dessiné passe au-dessus : un mot qui passe dans le coin ne doit pas masquer les vies.

Bravo, le cœur du jeu tourne ! Il ne manque plus que... pouvoir tirer. C'est le TD2.

4 Pour aller plus loin (si vous avez fini)

1. **Vitesses variées :** chaque mot reçoit une vitesse tirée au hasard entre 85 % et 115 % de `VITESSE_MOT`.
2. **Chronomètre :** affichez le temps de survie en haut à droite (un attribut `temps_jeu` augmenté de `dt`).
3. **Mots en danger :** un mot à moins de 150 px du joueur s'affiche en `ROUGE`.
4. **Invulnérabilité :** après un choc, le bouclier clignote pendant 1 s et ne peut pas être touché. *Indice :* un minuteur qui décroît, et `int(temps * 10) % 2` pour clignoter.
5. **Accélération :** la vitesse des nouveaux mots augmente de 2 px/s toutes les 10 secondes.

5 Backlog : tout ce qu'on peut ajouter au jeu

Dans un studio, la liste des fonctionnalités à faire s'appelle le *backlog*. Chaque tâche est petite, testable, et a une difficulté estimée (⋆ facile, ⋆⋆ moyen, ⋆⋆⋆ difficile). Les TP piochent dans cette liste ; cochez ce que vous avez réalisé. Les tâches marquées TP1, TP2, TP3 sont celles proposées en priorité à chaque séance.

Effets visuels (*game feel*)

☐ **Explosion de particules** ⋆⋆ TP1

À la destruction d'un mot, 20 à 40 petits points partent dans toutes les directions, ralentissent et s'effacent. Une classe `Particule` avec une durée de vie, comme `Laser`.

☐ **Tremblement d'écran** ⋆ TP1
Quand le bouclier est touché, tout le dessin est décalé d'un vecteur aléatoire qui diminue en 0,3 s.

☐ **Ciel en parallaxe** ⋆⋆ TP1
Trois couches d'étoiles qui défilent à des vitesses différentes donnent une impression de profondeur.

☐ **Nébuleuse et planètes** ⋆⋆
Un fond pré-dessiné une fois sur une `Surface` : dégradé, grands disques semi-transparents, planète avec anneau.

☐ **Canon qui tourne en douceur** ⋆⋆ TP1
Au lieu de pointer instantanément, le canon tourne vers sa cible à vitesse angulaire limitée (`Vector2.angle_to`).

☐ **Lettres qui volent** ⋆⋆
Chaque lettre tapée se détache du mot et tombe en tournant.

☐ **Bouclier qui clignote** ⋆
Une seconde d'invulnérabilité après un choc, avec clignotement.

☐ **Couleur du combo** ⋆
Le vaisseau et les lasers changent de couleur selon le multiplicateur.

☐ **Images (sprites)** ⋆
Remplacer le triangle par une image `.png` (`pygame.image.load`, `pygame.transform.rotate`).

Son

☐ **Bruitages** ⋆ TP1
Un son pour chaque frappe juste, faute, destruction, choc, niveau (`pygame.mixer.Sound`).

☐ **Musique de fond** ⋆
`pygame.mixer.music` en boucle, coupée dans le menu pause.

☐ **Sons synthétisés** ⋆⋆⋆
Générer les sons par calcul (sinusoïde, bruit) sans aucun fichier.

Mots et langue

☐ **Dictionnaire externe** ⋆ TP1
Charger les mots depuis `mots.txt` (`open`, `split`), avec la liste du code en secours si le fichier manque.

☐ **Accents** ⋆⋆ TP1
Accepter « é » et « e » (normalisation avec `unicodedata`), ou exiger l'accent en mode difficile.

☐ **Thèmes de mots** ⋆
Espace, Python, informatique, capitales… choisis dans le menu.

☐ **Majuscules** ⋆⋆
En mode expert, la casse compte (`Maj` obligatoire).

Ennemis

☐ **Mot rapide** ⋆ TP2
Court, deux fois plus rapide, affiché en jaune.

☐ **Mot blindé** ⋆⋆ TP2
Il faut le taper deux fois (un 2e mot apparaît quand le 1er est détruit).

☐ **Mot diviseur** ⋆⋆ TP2
En mourant, il libère deux petits mots à sa place (héritage : `class MotDiviseur(Mot)`).

☐ **Mot zigzag** ⋆⋆
Trajectoire sinusoïdale autour de sa direction (`math.sin(temps)`).

☐ **Mot fantôme** ⋆⋆
Il devient invisible quelques secondes : il faut s'en souvenir.

☐ **Boss-phrase** ⋆⋆⋆ TP2
Tous les 5 niveaux, un vaisseau géant affiche une *phrase* (avec espaces) et envoie des petits mots. Une barre de vie montre la progression.

Bonus

☐ **Mot bonus « soin »** ⋆ TP2
Vert ; le détruire rend une vie.

☐ **Mot bonus « bombe »** ⋆ TP2
Le détruire détruit tous les mots à l'écran.

☐ **Mot bonus « gel »** ⋆⋆
Ralentit tous les mots pendant 5 s (un facteur de temps appliqué à `dt`).

☐ **Mot bonus « bouclier »** ⋆⋆
Absorbe le prochain choc.

Progression

☐ **Vagues** ⋆⋆ TP2
Un nombre fixe de mots par vague, une pause entre deux vagues.

☐ **Améliorations entre les vagues** ⋆⋆⋆ TP2
Comme dans Glyphica : choisir 1 amélioration parmi 3 tirées au hasard (vie max, recul, multiplicateur, ralentissement…).

☐ **Modes de jeu** ⋆⋆
Survie (infini), Contre-la-montre (60 s), Zen (pas de vies).

☐ **Niveaux de difficulté** ⋆ TP3
Facile / Normal / Difficile modifient les constantes de difficulté.

Interface et données

☐ **Pause** ⋆ TP3
Un nouvel état `"pause"` dans la machine à états (touche `Tab`).

☐ **Meilleurs scores sauvegardés** ⋆⋆ TP3
Top 10 dans un fichier `scores.json` (`json.load`/`json.dump`), avec `try/except` si le fichier n'existe pas.

☐ **Saisie du pseudo** ⋆⋆ TP3
Un état `"saisie_nom"` après un record.

☐ **Statistiques détaillées** ⋆⋆ TP3
En fin de partie : lettres les plus ratées (dictionnaire de compteurs), meilleur mot par minute, histogramme dessiné avec `pygame.draw.rect`.

☐ **Menu d'options** ⋆⋆
Volume, plein écran, thème, navigation aux flèches.

☐ **Tutoriel** ⋆
Un premier niveau commenté pour les nouveaux joueurs.

Qualité du code

☐ **Docstrings et constantes** ⋆ TP3
Chaque classe et méthode documentée, aucun « nombre magique » hors des constantes.

☐ **Tests de la logique** ⋆⋆
Tester `Mot.taper`, `multiplicateur`, `precision` avec `assert`, sans ouvrir de fenêtre.

☐ **Mode debug** ⋆
`F1` affiche FPS, nombre de mots, cercles de collision.

Bilan de la séance

Ce qu'il faut retenir

- Un jeu = une **boucle** : événements → mise à jour → dessin, cadencée par `tick(FPS)`.
- On **redessine tout** à chaque image ; l'ordre de dessin définit ce qui est devant.
- Les vitesses s'expriment en **unités par seconde** et on multiplie par `dt`.
- Repère pygame : origine en haut à gauche, y **vers le bas**.
- Direction de A vers B : `(B - A).normalize()` ; mouvement : `p += v * dt`.
- Un **minuteur** = une variable qu'on augmente de `dt` et qu'on compare à un délai.
- On ne supprime **jamais** d'éléments d'une liste qu'on est en train de parcourir.
- On développe par **petites étapes testées**.

Pour le TD2 : gardez votre fichier ! Au TD2, on rendra le jeu *jouable* : taper les mots pour les détruire, lasers, score, menu et difficulté croissante.